RobiOmni ROS1 Chassis Package User Guide

This guide covers environment setup, USB permissions, building, startup, status monitoring, and control for the robiomni chassis communication package. Commands match the supplied ROS1 source package.

1 Tested Environment

Operating system: Ubuntu 20.04.6 LTS, ARM64 (aarch64)
ROS distribution: ROS Noetic
Package name: robiomni
Workspace archive: robiomni_ros1.zip

The workspace includes CAN libraries for ARM and x86, in 32-bit and 64-bit variants. CMake selects the library for the target architecture and compiler pointer size. Rebuild on the destination Linux/ROS system. ROS availability depends on the target platform. All topic prefixes now use robiomni; topic names remain case-sensitive. Custom message types use the robiomni package prefix.

2 USB Permissions for the CAN Adapter

Method 1: Create a udev rule. After running this command, reconnect the USB cable of the CAN adapter or restart the system.

```
sudo bash -c "echo 'ACTION==\"add\",SUBSYSTEMS==\"usb\", ATTRS{idVendor}==\"04d8\", ATTRS{idProduct}==\"0053\", GROUP=\"users\", MODE=\"0777\"' >> /etc/udev/rules.d/99-usbcan.rules"
```

Method 2: Set permissions manually. Run lsusb and find the Bus ID and Device ID of the Microchip Technology, Inc. device.

```
lsusb
sudo chmod 777 /dev/bus/usb/<BUS_ID>/<DEVICE_ID>
```

Example: sudo chmod 777 /dev/bus/usb/001/006. Substitute the actual IDs. Reconnecting or restarting is unnecessary after this manual change. Device IDs and permissions may change when the adapter is reconnected.

3 Prepare the ROS Workspace

Option 1: Extract the supplied workspace archive and enter its directory. The archive already contains src/robiomni, build.sh, and start.sh.

```
cd ~/ros_ws
```

Option 2: Create a workspace manually. Copy the robiomni package into src so that its manifest is located at src/robiomni/package.xml.

```
source /opt/ros/noetic/setup.bash
mkdir -p ~/ros_ws/src
cd ~/ros_ws/src
catkin_init_workspace
cd ..
```

4 Build the ROS Package

Open a terminal in the workspace root, load ROS, and install dependencies. If rosdep has not been initialized, first run sudo rosdep init and rosdep update.

```
source /opt/ros/noetic/setup.bash
rosdep install --from-paths src --ignore-src -r -y --rosdistro noetic
catkin_make -j2
```

With the complete workspace archive, you can also run bash build.sh from the workspace root.

5 Load the Environment

In every new terminal, enter the workspace root and run:

```
source /opt/ros/noetic/setup.bash
source ./devel/setup.bash
```

6 Start the ROS Package

Connect the CAN adapter, configure USB permissions, and run:

```
roslaunch robiomni robiomni.launch
```

With the complete workspace archive, you can also run bash start.sh. The launch filename is robiomni.launch. The executable is robiomni and the ROS node name is robiomni_node.

7 Monitor Published Topics

After starting the node, run these commands in another terminal with the workspace environment loaded. All four original monitoring functions are retained, together with the additional interfaces provided by the current package. Live values require CAN feedback from the chassis.

Read motion feedback:

```
rostopic echo /robiomni_Motion
```

Read system status:

```
rostopic echo /robiomni_System
```

Read drive motor status:

```
rostopic echo /robiomni_Drive_Motor
```

Read steering motor status:

```
rostopic echo /robiomni_Steering_Motor
```

Read remote control joystick status:

```
rostopic echo /robiomni_RemoteCtrl_Rocker
```

Read remote control key status:

```
rostopic echo /robiomni_RemoteCtrl_Key
```

Read automatic charging status:

```
rostopic echo /robiomni_Charge_State
```

Read drive motor current:

```
rostopic echo /robiomni_Drive_Motor_Current
```

Read drive motor faults:

```
rostopic echo /robiomni_Drive_Motor_Error
```

Read steering motor current:

```
rostopic echo /robiomni_Steering_Motor_Current
```

Read steering motor faults:

```
rostopic echo /robiomni_Steering_Motor_Error
```

Read system status group 2:

```
rostopic echo /robiomni_System_2
```

Read odometry:

```
rostopic echo /robiomni_Odom_Info
```

Read standard battery status:

```
rostopic echo /robiomni_Battery_State
```

Read bms information:

```
rostopic echo /robiomni_BMS_Info
```

Read emergency stop button status:

```
rostopic echo /robiomni/std_stop_button_info
```

Read software stop status:

```
rostopic echo /robiomni/std_software_stop_info
```

Read front collision status:

```
rostopic echo /robiomni/std_front_collision_info
```

Read rear collision status:

```
rostopic echo /robiomni/std_rear_collision_info
```

Inspect available topics, message types, and publication rates:

```
rostopic list

rostopic type /robiomni_Motion

rostopic hz /robiomni_Motion

rosmsg show robiomni/ht_motion
```

8 Control the Chassis with Topics

The current /robiomni_Control subscriber accepts geometry_msgs/Twist, not robiomni/ht_control. Although that custom message file exists, the node does not subscribe to /robiomni_Control using it. The commands below use the interfaces subscribed to by the current source.

Omnidirectional FTFD: straight travel at 0.1 m/s. Set linear.x=0.1 and angular.z=0.

```
rostopic pub -r 50 /robiomni_FTFD_Ctrl geometry_msgs/Twist '{"linear":{"x":0.1,"y":0,"z":0},"angular":{"x":0,"y":0,"z":0}}'
```

Turn while moving: linear.x=0.1 m/s and angular.z=0.1 rad/s. angular.z is angular velocity in this mode; linear.y controls lateral velocity in m/s.

```
rostopic pub -r 50 /robiomni_FTFD_Ctrl geometry_msgs/Twist '{"linear":{"x":0.1,"y":0,"z":0},"angular":{"x":0,"y":0,"z":0.1}}'
```

Stop this mode: stop the previous publisher, then continuously publish zero values on the same topic. Confirm the chassis has stopped before pressing Ctrl+C.

```
rostopic pub -r 50 /robiomni_FTFD_Ctrl geometry_msgs/Twist '{"linear":{"x":0,"y":0,"z":0},"angular":{"x":0,"y":0,"z":0}}'
```

Ackermann: straight travel at 0.1 m/s. Set linear.x=0.1 and angular.z=0.

```
rostopic pub -r 50 /robiomni_Ackermann_Ctrl geometry_msgs/Twist '{"linear":{"x":0.1,"y":0,"z":0},"angular":{"x":0,"y":0,"z":0}}'
```

Turn while moving: linear.x=0.1 m/s and angular.z=0.1 rad (about 5.7 degrees). angular.z is the steering angle, not angular velocity; keep linear.y=0. This is not rotation in place.

```
rostopic pub -r 50 /robiomni_Ackermann_Ctrl geometry_msgs/Twist '{"linear":{"x":0.1,"y":0,"z":0},"angular":{"x":0,"y":0,"z":0.1}}'
```

Stop this mode: stop the previous publisher, then continuously publish zero values on the same topic. Confirm the chassis has stopped before pressing Ctrl+C.

```
rostopic pub -r 50 /robiomni_Ackermann_Ctrl geometry_msgs/Twist '{"linear":{"x":0,"y":0,"z":0},"angular":{"x":0,"y":0,"z":0}}'
```

General motion control through /robiomni_Control: A pure angular velocity command rotates in place; other inputs use omnidirectional control.

```
rostopic pub -r 50 /robiomni_Control geometry_msgs/Twist '{"linear":{"x":0.1,"y":0,"z":0},"angular":{"x":0,"y":0,"z":0}}'
```

Double Ackermann: straight travel at 0.1 m/s. Set linear.x=0.1 and angular.z=0.

```
rostopic pub -r 50 /robiomni_Double_Ackermann_Ctrl geometry_msgs/Twist '{"linear":{"x":0.1,"y":0,"z":0},"angular":{"x":0,"y":0,"z":0}}'
```

Turn while moving: linear.x=0.1 m/s and angular.z=0.1 rad (about 5.7 degrees). angular.z is the steering angle, not angular velocity; keep linear.y=0. This is not rotation in place.

```
rostopic pub -r 50 /robiomni_Double_Ackermann_Ctrl geometry_msgs/Twist '{"linear":{"x":0.1,"y":0,"z":0},"angular":{"x":0,"y":0,"z":0.1}}'
```

Stop this mode: stop the previous publisher, then continuously publish zero values on the same topic. Confirm the chassis has stopped before pressing Ctrl+C.

```
rostopic pub -r 50 /robiomni_Double_Ackermann_Ctrl geometry_msgs/Twist '{"linear":{"x":0,"y":0,"z":0},"angular":{"x":0,"y":0,"z":0}}'
```

Rotation in place: use this separate topic with linear.x=linear.y=0 and angular.z=0.1 rad/s. Here angular.z is angular velocity; this is separate from the Ackermann steering-angle command.

```
rostopic pub -r 50 /robiomni_Rotate_Ctrl geometry_msgs/Twist '{"linear":{"x":0,"y":0,"z":0},"angular":{"x":0,"y":0,"z":0.1}}'
```

Stop motion by continuously publishing zero velocity to the active control topic. The example below uses the omnidirectional interface. After sending the stop command, press Ctrl+C to end publication.

```
rostopic pub -r 50 /robiomni_FTFD_Ctrl geometry_msgs/Twist '{"linear":{"x":0,"y":0,"z":0},"angular":{"x":0,"y":0,"z":0}}'
```

Turn the front light off and on. The current interface is /robiomni_Lamp1_Ctrl; the /robiomni_Lamp topic in the original guide is no longer applicable.

```
rostopic pub -1 /robiomni_Lamp1_Ctrl std_msgs/Bool '{"data":false}'

rostopic pub -1 /robiomni_Lamp1_Ctrl std_msgs/Bool '{"data":true}'
```

Turn all lights off and on:

```
rostopic pub -1 /robiomni_Lamp_All_Ctrl std_msgs/Bool '{"data":false}'

rostopic pub -1 /robiomni_Lamp_All_Ctrl std_msgs/Bool '{"data":true}'
```

Turn the rear light off and on:

```
rostopic pub -1 /robiomni_Lamp2_Ctrl std_msgs/Bool '{"data":false}'

rostopic pub -1 /robiomni_Lamp2_Ctrl std_msgs/Bool '{"data":true}'
```

Disable automatic charging and enable infrared charging:

```
rostopic pub -1 /robiomni_Charge_Ctrl std_msgs/UInt8 '{"data":0}'

rostopic pub -1 /robiomni_Charge_Ctrl std_msgs/UInt8 '{"data":1}'
```

Clear collision status and reset odometry:

```
rostopic pub -1 /robiomni_Collision_Clean std_msgs/Empty '{}'

rostopic pub -1 /robiomni_Odom_Clean std_msgs/Empty '{}'
```

The unified motion interface /robiomni_Motion_Ctrl uses robiomni/ht_motion. Inspect its fields and confirm mode values, units, and ranges against the chassis protocol before use. Do not use status feedback fields directly as control values.

```
rosmsg show robiomni/ht_motion
```